Microservicios y APIs Escalables

Universidad de Palermo · 2026

Sprint 14

Seguridad: Autenticación con JWT

ms-auth · JJWT 0.12.x · Spring Cloud Gateway Filter

Objetivo del sprint

En este sprint vas a implementar autenticación con **JWT (JSON Web Token)** en el sistema "Patitas Felices". Esto resuelve el problema de que cualquier persona puede llamar a los microservicios sin identificarse.

La arquitectura de seguridad que vas a construir tiene tres partes:

- **ms-auth** (nuevo microservicio en :8085): emite y valida tokens. El único que conoce la clave secreta y tiene la lógica de seguridad.
- **API Gateway** (actualización): intercepta TODAS las peticiones con un filtro global, valida el token JWT y rechaza las no autenticadas antes de que lleguen a los microservicios de negocio.
- **Microservicios de negocio** (sin cambios): no validan tokens. Solo confían en que el Gateway ya filtró las peticiones no autorizadas.

> 📌 **Flujo completo de autenticación**
>
> REGISTRO: POST /auth/register → ms-auth crea usuario con contraseña encriptada
> LOGIN: POST /auth/login → ms-auth valida contraseña, devuelve JWT
> PETICIÓN: GET /api/mascotas → Gateway valida JWT en header Authorization
> Si válido → pasa la petición al microservicio
> Si inválido/ausente → HTTP 401 Unauthorized

Marco conceptual

¿Qué es JWT?

JWT es un **token con tres partes separadas por puntos**: Header.Payload.Signature. El servidor firma el token con una clave secreta, y cualquiera que tenga esa clave puede verificar que no fue modificado, sin necesidad de consultar una base de datos.

Parte	Contenido	Ejemplo
Header	Algoritmo de firma	{"alg":"HS256","typ":"JWT"}
Payload (Claims)	Datos del usuario (sub, rol, exp)	{"sub":"jorge","rol":"ADMIN","exp"

Parte	Contenido	Ejemplo
		:1234567890}
Signature	HMAC-SHA256(base64(header) +"."+base64(payload), secret)	dBjftJeZ4CVP-mB92...

Por qué el Gateway valida y no cada microservicio

Enfoque	Ventajas	Desventajas
Gateway valida (este sprint)	Un solo punto de validación. Los microservicios no necesitan Spring Security. Cambiar la lógica de autenticación no requiere redesplegar todos los servicios.	Si el Gateway falla, no hay autenticación.
Cada microservicio valida	Más robusto si el Gateway falla.	Duplicación de código. Todos los microservicios necesitan la clave JWT. Mayor acoplamiento.

PASO 1 — Crear el módulo ms-auth

Creá un nuevo módulo Maven en el proyecto padre llamado `ms-auth`. Puerto: 8085.

Dependencias en pom.xml de ms-auth:

```
<!-- Spring Boot Web + JPA + MySQL (igual que los otros microservicios) -->
<dependency>spring-boot-starter-web</dependency>
<dependency>spring-boot-starter-data-jpa</dependency>
<dependency>mysql-connector-j</dependency>

<!-- Spring Security -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- JJWT (JSON Web Token) -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.6</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
```

```
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>

<!-- Eureka Client -->
<dependency>spring-cloud-starter-netflix-eureka-client</dependency>
```

application.yml de ms-auth (campos principales): puerto 8085, nombre `ms-auth`, conexión MySQL a base de datos `patitas_auth`, url Eureka, y la clave JWT (mínimo 256 bits / 32 caracteres).

> ⚠️ **La clave JWT debe tener mínimo 256 bits**
> JJWT 0.12.x requiere claves de al menos 256 bits para HS256.
> 256 bits = 32 bytes = 32 caracteres ASCII.
> Usá una clave larga como: "patitas-felices-secret-key-2026-muy-larga-para-ser-segura"
> Esta MISMA clave debe estar también en el application.yml del api-gateway.

PASO 2 — Crear la entidad Usuario

Paquete: ms-auth → domain/model

Creá la entidad `Usuario` con los siguientes campos:

```
// Entidad Usuario:
// - id: Long (GeneratedValue AUTO)
// - username: String (Column unique=true, nullable=false)
// - password: String (la guardamos encriptada con BCrypt)
// - rol: String (ej: "ADMIN", "VET", "RECEPCIONISTA")

// Anotaciones de clase:
// @Entity, @Table(name = "usuarios")
```

Creá también `UsuarioJpaRepository` que extiende JpaRepository<Usuario, Long> con un método:

```
Optional<Usuario> findByUsername(String username);
```

Creá una base de datos MySQL llamada `patitas_auth` (igual que las otras: cada microservicio tiene su propia BD).

```
-- En MySQL:
CREATE DATABASE patitas_auth;
```

PASO 3 — Implementar JwtService

Paquete: ms-auth → application/service

Creá la clase `JwtService` anotada con `@Service`. Esta clase tiene toda la lógica JWT. Necesita tres métodos principales:

Método 1 — generateToken(String username, String rol):

```
// Usa Jwts.builder()
// .subject(username)
// .claim("rol", rol)
// .issuedAt(new Date())
// .expiration(new Date(System.currentTimeMillis() + 86400000)) // 24hs
// .signWith(getSigningKey())
// .compact()
```

Método 2 — validateToken(String token):

```
// Usa Jwts.parser()
// .verifyWith(getSigningKey())
// .build()
// .parseSignedClaims(token)
// Si no lanza excepción → token válido → retorna true
// Si lanza JwtException → token inválido → retorna false
```

Método 3 — extractUsername(String token):

```
// Parsea el token y retorna getClaims().getSubject()
```

Método privado getSigningKey():

```
// Lee la clave del application.yml con @Value("${jwt.secret}")
// Convierte a SecretKey con:
// Keys.hmacShaKeyFor(secretKey.getBytes(StandardCharsets.UTF_8))
```

> 📌 **Por qué tres dependencias JJWT (api, impl, jackson)**
> jjwt-api: las interfaces y clases públicas (lo que usás en el código)
> jjwt-impl: la implementación en runtime (scope runtime, no se compila contra ella)
> jjwt-jackson: serialización JSON del payload (scope runtime también)
> Esta separación permite cambiar la implementación sin cambiar tu código.

PASO 4 — Implementar AuthService y AuthController

Paquete: ms-auth → application/service y adapters/in/web

AuthService — lógica de negocio de autenticación:

```
// Método register(RegisterRequest request):
```

```
  // 1. Crear nuevo Usuario
  // 2. Encodear contraseña: passwordEncoder.encode(request.password())
  // 3. Guardar en repositorio
  // 4. Retornar AuthResponse con el token generado

  // Método login(LoginRequest request):
  // 1. Buscar usuario por username (lanzar excepción si no existe)
  // 2. Verificar contraseña: passwordEncoder.matches(request.password(),
usuario.getPassword())
  // 3. Si no coincide → lanzar CredencialesInvalidasException
  // 4. Generar y retornar token JWT
```

Records/DTOs necesarios: `RegisterRequest(String username, String password, String rol)`, `LoginRequest(String username, String password)`, `AuthResponse(String token)`.

AuthController — endpoints REST:

```
  // @RestController
  // @RequestMapping("/auth")

  // POST /auth/register → authService.register(request) → 200 OK + AuthResponse
  // POST /auth/login    → authService.login(request)    → 200 OK + AuthResponse
```

SecurityConfig — deshabilitar Spring Security por defecto (ms-auth emite tokens, no los valida con filtros):

```
  // @Configuration @EnableWebSecurity
  // SecurityFilterChain: desactivar CSRF, permitir todos los endpoints de /auth/**
  // Exponer BCryptPasswordEncoder como @Bean
  // Exponer AuthenticationManager como @Bean
```

PASO 5 — Crear AuthenticationFilter en el API Gateway

Módulo: api-gateway · paquete filter

Este filtro es el núcleo de la seguridad. Intercepta TODAS las peticiones entrantes al Gateway. Implementa `GlobalFilter` y `Ordered` (Spring Cloud Gateway es reactivo — WebFlux).

```
  // Dependencia en api-gateway pom.xml:
  <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-api</artifactId>
      <version>0.12.6</version>
  </dependency>
  <dependency>
      <groupId>io.jsonwebtoken</groupId>
      <artifactId>jjwt-impl</artifactId>
      <version>0.12.6</version>
```

```
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
</dependency>
```

Lógica del filtro (método filter): recibe `ServerWebExchange exchange` y `GatewayFilterChain chain`.

```
// 1. Obtener la ruta de la petición
//    String path = exchange.getRequest().getPath().value();

// 2. Si la ruta empieza con "/auth/" → dejar pasar sin validar
//    if (path.startsWith("/auth/")) return chain.filter(exchange);

// 3. Obtener el header Authorization
//    String authHeader = exchange.getRequest()
//        .getHeaders().getFirst("Authorization");

// 4. Si no tiene header o no empieza con "Bearer " → HTTP 401
//    exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED);
//    return exchange.getResponse().setComplete();

// 5. Extraer el token (quitar el prefijo "Bearer ")
//    String token = authHeader.substring(7);

// 6. Validar el token con jwtService.validateToken(token)
//    Si inválido → HTTP 401 + return

// 7. Si válido → agregar username como header para downstream
//    ServerHttpRequest request = exchange.getRequest().mutate()
//        .header("X-Auth-User", jwtService.extractUsername(token))
//        .build();
//    return chain.filter(exchange.mutate().request(request).build());
```

⚠️ **API Gateway usa WebFlux — NO usar HttpServletRequest**

Spring Cloud Gateway es reactivo (WebFlux), NO servlet.
NO usar: HttpServletRequest, HttpServletResponse, FilterChain (javax/jakarta)
SÍ usar: ServerWebExchange, GatewayFilterChain, ServerHttpRequest
La respuesta 401 es: exchange.getResponse().setStatusCode(HttpStatus.UNAUTHORIZED)
return exchange.getResponse().setComplete()

PASO 6 — Configurar la ruta /auth/** en el Gateway

Archivo: api-gateway/src/main/resources/application.yml

Agregá la ruta para ms-auth y la clave JWT al application.yml del Gateway:

Ruta para ms-auth:

```
spring:
  cloud:
    gateway:
      routes:
        # ... rutas existentes ...
        - id: ms-auth
          uri: lb://ms-auth
          predicates:
            - Path=/auth/**
          # Sin filtros de autenticación — la ruta pública de login/registro
```

Clave JWT (debe ser idéntica a la de ms-auth):

```
jwt:
  secret: patitas-felices-secret-key-2026-muy-larga-para-ser-segura
```

📌 **La clave JWT debe ser IDÉNTICA en ms-auth y api-gateway**

ms-auth usa la clave para FIRMAR el token (generateToken).
api-gateway usa la clave para VERIFICAR la firma (validateToken).
Si las claves son diferentes, la verificación siempre falla con HTTP 401.
En producción, esta clave viene de un Config Server o un gestor de secretos.

El `AuthenticationFilter` ya excluye las rutas `/auth/**` en su lógica interna (Paso 5). La ruta de Gateway solo asegura que las peticiones a `/auth/**` se enruten a ms-auth.

PASO 7 — Registrar ms-auth en Eureka y Config Server

ms-auth debe estar registrado en Eureka para que el Gateway pueda resolverlo por nombre (`lb://ms-auth`).

application.yml de ms-auth — sección Eureka:

```
spring:
  application:
    name: ms-auth

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true
```

Orden de arranque del sistema completo:

Orden	Servicio	Puerto	Por qué primero
1	config-server	:8888	Los demás leen su configuración de aquí
2	eureka-server	:8761	Requiere config-server para su configuración
3	ms-auth	:8085	Necesita Eureka para registrarse
4	ms-dueno, ms-mascota, ms-veterinario, ms-turno	:8081-84	Necesitan Eureka
5	api-gateway	:8080	Necesita Eureka para resolver todos los servicios

> 📌 **Si no usás Config Server todavía**
> Si ms-auth tiene todo en su propio application.yml (sin config-server),
> el orden es simplemente: eureka-server → ms-auth → demás microservicios → gateway.

PASO 8 — Verificación con 5 escenarios

Verificá el sistema completo con estos escenarios en Postman:

Escenario 1 — Registro de usuario:

```
POST http://localhost:8080/auth/register
Body: {"username": "jorge", "password": "1234", "rol": "ADMIN"}
Resultado esperado: HTTP 200 + {"token": "eyJhbGc..."}
```

Escenario 2 — Login:

```
POST http://localhost:8080/auth/login
Body: {"username": "jorge", "password": "1234"}
Resultado esperado: HTTP 200 + {"token": "eyJhbGc..."}
(Copiá el token — lo usás en los siguientes escenarios)
```

Escenario 3 — Petición autenticada:

```
GET http://localhost:8080/api/mascotas
Header: Authorization: Bearer eyJhbGc...
Resultado esperado: HTTP 200 + lista de mascotas
```

Escenario 4 — Petición sin token:

```
GET http://localhost:8080/api/mascotas
(Sin header Authorization)
Resultado esperado: HTTP 401 Unauthorized
(El Gateway rechaza antes de llegar a ms-mascota)
```

Escenario 5 — Token inválido o expirado:

```
GET http://localhost:8080/api/mascotas
Header: Authorization: Bearer eyJhbGcTokenFalso
Resultado esperado: HTTP 401 Unauthorized
```

PASO 9 — Mejoras opcionales para explorar

El sprint base está completo. Si querés extenderlo, estas son las mejoras más comunes:

Opción A — Pasar el usuario a los microservicios de negocio:

- El Gateway ya agrega el header `X-Auth-User` con el username al pasar la petición (Paso 5, Paso 7 del filter).
- En los microservicios de negocio, podés recibir ese header con `@RequestHeader("X-Auth-User") String username` en los controllers.
- Útil para auditoría: saber qué usuario creó o modificó un turno.

Opción B — Autorización por rol:

- En el `AuthenticationFilter`, después de validar el token, extraer el claim "rol".
- Si la ruta requiere un rol específico (ej: solo ADMIN puede crear veterinarios), verificar el claim antes de dejar pasar.
- Agregar el rol también como header downstream: `X-Auth-Rol`.

Opción C — Refresh Token:

- El token actual expira en 24hs. Un refresh token permite obtener un nuevo token sin hacer login con contraseña.
- Agregar un nuevo endpoint `POST /auth/refresh` que reciba el refresh token (guardado en BD) y devuelva un nuevo access token.

> 📌 **¿Cuál implementar primero?**
> Para el TP: la Opción A (X-Auth-User) es la más sencilla y agrega valor real.
> La Opción B (autorización por rol) es más compleja pero completa el modelo de seguridad.
> La Opción C (refresh token) requiere más cambios en la BD y es opcional para el TP.

Resumen del sprint

Paso	Qué hiciste	Módulo
1	Módulo ms-auth con dependencias Spring Security + JJWT + Eureka	ms-auth (nuevo)

Paso	Qué hiciste	Módulo
2	Entidad Usuario (username, password BCrypt, rol) + UsuarioJpaRepository	ms-auth / domain
3	JwtService: generateToken, validateToken, extractUsername	ms-auth / application
4	AuthService (register + login) + AuthController (/auth/register, /auth/login)	ms-auth / application + adapters
5	AuthenticationFilter (GlobalFilter reactivo) en Gateway	api-gateway / filter
6	Ruta /auth/** en Gateway + clave JWT compartida	api-gateway / application.yml
7	ms-auth en Eureka + orden de arranque	Configuración general
8	5 escenarios de verificación en Postman	Testing
9	Mejoras opcionales: X-Auth-User, autorización por rol, refresh token	Extensiones

📌 **Reglas de este sprint**

- La clave JWT debe ser IDÉNTICA en ms-auth y api-gateway. Mínimo 32 caracteres.
- Spring Cloud Gateway usa WebFlux: NO HttpServletRequest, SÍ ServerWebExchange.
- Los microservicios de negocio (ms-mascota, etc.) NO validan JWT — solo el Gateway.
- Las rutas /auth/** deben estar excluidas del filtro (para que el login funcione).
- JJWT necesita tres dependencias: jjwt-api (compile), jjwt-impl, jjwt-jackson (runtime).